

Databricks Certified Data Engineer Associate — Prep Sheet

Exam domains, sample questions with answers, and gotchas

Lakehouse · ELT (Spark SQL/Python) · incremental (Delta/MERGE/Auto Loader/DLT) · pipelines · governance

Topics, sample questions with answers, and gotchas for the **Databricks Certified Data Engineer Associate** exam. (Always confirm the latest exam guide on Databricks' site — details can change.)

Exam at a glance

- ~45 multiple-choice questions, 90 minutes, online proctored.
- Passing score ~70%. No hard prerequisites; ~6 months of Databricks experience recommended.
- Focus: **Spark SQL & PySpark on the lakehouse, Delta Lake, incremental processing, production pipelines, and basic governance.**

The 5 domains (study weight)

1. **Databricks Lakehouse Platform** (~24%) — architecture, clusters, notebooks, Repos, DBFS, Delta basics.
 2. **ELT with Spark SQL & Python** (~29%) — extract, transform, create tables/views, CTAS, Delta writes.
 3. **Incremental Data Processing** (~22%) — Delta, MERGE, COPY INTO, Auto Loader, Structured Streaming, DLT.
 4. **Production Pipelines** (~16%) — Jobs/Workflows, DLT pipelines, alerts, scheduling.
 5. **Data Governance** (~9%) — Unity Catalog, permissions, PII/basic security.
-

DOMAIN 1 — Lakehouse Platform (know cold)

- **Lakehouse** = data lake storage + warehouse reliability (Delta) on one platform.
- **Cluster** = driver + workers; **all-purpose** (interactive) vs **job** clusters (per run, cheaper).
- **Delta Lake** = Parquet + transaction log → ACID, time travel, schema enforcement.
- **Repos** = Git integration; **DBFS** = filesystem over cloud storage.
- **Managed vs external tables**: managed → Databricks owns data+metadata (drop deletes data); external (with `LOCATION`) → Databricks owns only metadata (drop keeps files).

DOMAIN 2 — ELT with Spark SQL & Python

- **Create table as select (CTAS)**: `CREATE TABLE t AS SELECT ...`.
- **Read files directly**: `SELECT * FROM parquet.`/path` / json.`/path``.
- **Views**: `CREATE VIEW` (logical), **temp view** (session), **global temp view** (cluster).
- **Delta write modes**: `overwrite`, `append`; `INSERT OVERWRITE`, `INSERT INTO`.
- **Handling nested/JSON**: `:` and `.` access, `explode()`, `from_json()`.
- Deduplicate with `ROW_NUMBER()` window; joins; `MERGE`.

DOMAIN 3 — Incremental Processing (the heaviest technical area)

- **MERGE** = upsert (insert/update/delete atomically) — the core incremental pattern.
- **COPY INTO** = idempotent, retrieable bulk load from files into a Delta table (skips already-loaded).
- **Auto Loader** (`cloudFiles`) = incremental *streaming* file ingestion; tracks new files via a checkpoint; schema inference/evolution.
- **Structured Streaming** = `readStream` → transform → `writeStream` with a **checkpoint** (exactly-once); triggers: `availableNow` , `processingTime` .
- **OPTIMIZE** (compact small files) + **ZORDER** (data skipping); **VACUUM** (remove old files, default 7-day retention).
- **Time travel**: `VERSION AS OF` / `TIMESTAMP AS OF` .

DOMAIN 4 — Production Pipelines

- **Jobs/Workflows**: tasks, dependencies (DAG), schedules (cron), retries, email alerts, job clusters.
- **Delta Live Tables (DLT)**: declarative pipelines; `@dlt.table` ; **expectations** (`@dlt.expect` , `expect_or_drop` , `expect_or_fail`) for data quality; auto lineage.
- **Streaming vs batch tables in DLT**; `STREAMING LIVE TABLE` .

DOMAIN 5 — Governance (Unity Catalog)

- Hierarchy: **metastore** → **catalog** → **schema** → **table**; reference as `catalog.schema.table` .
- **GRANT/REVOKE** privileges (`USE CATALOG/SCHEMA` , `SELECT` , `MODIFY`) to **groups**.
- Column/row security via **dynamic views**; lineage + audit are automatic.

SAMPLE QUESTIONS (with answers)

1. You drop a **managed** Delta table. What happens to the data? A) Nothing B) Only metadata removed **C) Both metadata and underlying data are deleted** D) Moved to trash → **C**. Managed tables own the data; external tables (`LOCATION`) keep the files on drop.
2. Which command performs an idempotent bulk load that skips files already loaded? A) `INSERT INTO` **B) COPY INTO** C) `MERGE` D) `CREATE TABLE AS` → **B**. `COPY INTO` tracks loaded files and is retrieable/idempotent.
3. You need to upsert changes (update existing, insert new) into a Delta table. Use: **A) MERGE INTO** B) `INSERT OVERWRITE` C) `UPDATE` D) `COPY INTO` → **A**. `MERGE` = upsert.
4. Auto Loader is best described as: A) A batch CSV reader **B) Incremental streaming ingestion of new files with checkpointing** C) A cluster type D) A governance tool → **B**. `cloudFiles` incrementally ingests new files.

5. Which guarantees exactly-once processing in Structured Streaming? A) `mode("overwrite")` B) **A checkpoint location** C) `cache()` D) `OPTIMIZE` → **B**. Checkpointing tracks progress for exactly-once.
6. To compact many small files in a Delta table: A) `OPTIMIZE` B) `VACUUM` C) `REFRESH` D) `ANALYZE` → **A**. `OPTIMIZE` compacts; **ZORDER** adds data skipping; `VACUUM` only removes old files.
7. Query a Delta table as it was 3 versions ago: *A) `SELECT FROM t VERSION AS OF <v> * B) SELECT FROM t@3 C) ROLLBACK D) not possible → A. Time travel via VERSION AS OF / TIMESTAMP AS OF.`
8. In DLT, which expectation **drops** violating rows but keeps the pipeline running? A) `@dlt.expect` B) `@dlt.expect_or_drop` C) `@dlt.expect_or_fail` D) `@dlt.table` → **B**. `expect` warns; `expect_or_drop` drops bad rows; `expect_or_fail` fails the run.
9. Fastest way to avoid shuffling a large table when joining a small lookup: A) `repartition` B) **broadcast the small table** C) `cache` D) `coalesce` → **B**. Broadcast join sends the small table to all executors.
10. Which cluster type is cheapest for a scheduled nightly job? A) All-purpose B) **Job cluster** C) SQL warehouse D) Pool → **B**. Job clusters spin up per run and terminate — cheaper than always-on.
11. Grant read access to a table for a group in Unity Catalog: A) `GRANT SELECT ON TABLE cat.sch.t TO group` B) `ALTER` C) `USE` D) `SHOW GRANTS` → **A**.
12. A **temp view** is visible: A) To all clusters B) **Only within the current session/notebook** C) Forever D) Across workspaces → **B**. Temp views are session-scoped; global temp views are cluster-scoped.
13. Read a Parquet file directly with SQL: *A) `SELECT FROM parquet. /path/` B) LOAD parquet C) IMPORT D) COPY → A. ** File-source SQL: parquet. /path , json. ...`, etc.`
14. What does `VACUUM` do (and the default retention)? → Removes data files no longer referenced by the Delta log; **default 7-day** retention (protects time travel — don't lower without care).
15. Which is NOT a benefit of Delta Lake? A) ACID B) Time travel C) Schema enforcement D) **Sub-millisecond OLTP lookups** → **D**. Delta is analytics, not OLTP.
-

COMMON GOTCHAS (exam traps)

- **Managed vs external drop** — managed drop deletes data; external keeps files. (Very common.)
- **COPY INTO vs MERGE vs INSERT** — `COPY INTO` = idempotent bulk file load; `MERGE` = upsert; `INSERT` = plain append/overwrite.
- **OPTIMIZE vs VACUUM** — `OPTIMIZE` compacts; `VACUUM` deletes old files. Don't mix them up.
- **Auto Loader ≠ COPY INTO** — Auto Loader is *streaming/incremental*; `COPY INTO` is batch (though also incremental/idempotent).
- **Checkpoint required** for streaming exactly-once.

- **DLT expectations** — `expect` (warn) vs `expect_or_drop` (drop) vs `expect_or_fail` (fail).
- **Temp vs global temp vs permanent view** scope.
- **Job cluster vs all-purpose** for cost questions.
- **Unity Catalog 3-level namespace** `catalog.schema.table`; grant to **groups**.
- **Time travel syntax** `VERSION AS OF` / `TIMESTAMP AS OF`.

STUDY PLAN (1–2 weeks)

- Take the official **Data Engineering with Databricks** learning path (free self-paced).
- Do the companion notebook (`databricks_de_practice.py`) hands-on — write Delta, MERGE, OPTIMIZE, time travel, Auto Loader, a DLT pipeline, and a Job.
- Memorize the **gotchas** above and the domain keywords.
- Take a practice test; review every miss.
- On exam day: read carefully (many questions hinge on one keyword — managed/external, drop, OPTIMIZE/VACUUM).